

SIMATS ENGINEERING
CO3_ASSESSMENT TOOL3_Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192424390
Name	Venu G

COURSE OUTCOME COVERED

CO3: Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: $(P \vee Q) \Rightarrow R$, $R \Rightarrow \neg S$, $S \vee T$, $\neg T \Rightarrow P$. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of $\vee$ over $\wedge$). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α . Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of $KB \wedge \neg \alpha$, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) $Knows(John, x)$ and $Knows(John, Jane)$; (ii) $Knows(x, Elizabeth)$ and $Knows(y, x)$; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I _____ Venu G _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	10
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10

Q1. Propositional Logic to CNF Conversion

Knowledge base sentences:

- S1: $(P \vee Q) \Rightarrow R$
- S2: $R \Rightarrow \neg S$
- S3: $S \vee T$
- S4: $\neg T \Rightarrow P$

(a) Manual CNF Derivation (step by step)

Sentence 1: $(P \vee Q) \Rightarrow R$

Step 0 (Original): $(P \vee Q) \Rightarrow R$

Step 1 (Eliminate implication, $A \Rightarrow B \equiv \neg A \vee B$): $\neg(P \vee Q) \vee R$

Step 2 (Push negation inward, De Morgan $\neg(A \vee B) \equiv \neg A \wedge \neg B$): $(\neg P \wedge \neg Q) \vee R$

Step 3 (Distribute $\vee$ over $\wedge$, $(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$): $(\neg P \vee R) \wedge (\neg Q \vee R)$

CNF result: $(\neg P \vee R) \wedge (\neg Q \vee R) \Rightarrow$ clauses $\{\neg P, R\}$ and $\{\neg Q, R\}$

Sentence 2: $R \Rightarrow \neg S$

Step 0 (Original): $R \Rightarrow \neg S$

Step 1 (Eliminate implication): $\neg R \vee \neg S$

Step 2 (Negation already innermost — no change): $\neg R \vee \neg S$

Step 3 (Already a single disjunction of literals — no distribution needed): $\neg R \vee \neg S$

CNF result: $(\neg R \vee \neg S) \Rightarrow$ clause $\{\neg R, \neg S\}$

Sentence 3: $S \vee T$

Step 0 (Original): $S \vee T$

Step 1 (No implication to eliminate): $S \vee T$

Step 2 (No negation to push): $S \vee T$

Step 3 (Already a disjunction of literals): $S \vee T$

CNF result: $(S \vee T) \Rightarrow$ clause $\{S, T\}$

Sentence 4: $\neg T \Rightarrow P$

Step 0 (Original): $\neg T \Rightarrow P$

Step 1 (Eliminate implication): $\neg(\neg T) \vee P$

Step 2 (Double-negation elimination $\neg\neg T \equiv T$): $T \vee P$

Step 3 (Already a disjunction of literals): $T \vee P$

CNF result: $(T \vee P) \Rightarrow$ clause $\{T, P\}$

Combined KB in CNF (conjunction of all clauses):

$(\neg P \vee R) \wedge (\neg Q \vee R) \wedge (\neg R \vee \neg S) \wedge (S \vee T) \wedge (T \vee P)$

(b) Program (Python) to convert propositional sentences to CNF

The program below implements a small recursive-descent parser for propositional sentences (using $\sim$, $\&$, $|$, $>$, $=$ for $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$) and applies the three CNF-conversion steps (eliminate implications/biconditionals $\rightarrow$ push negation inward (NNF) $\rightarrow$ distribute $\vee$ over $\wedge$) exactly as done by hand above.

Program:

```
import re
def eliminate_implication(expr):
    while ">" in expr:
        match = re.search(r'(\([^()]+\))\s*\>\s*(\([^()]+\))\s*[A-Za-z~]', expr)
        if not match:
            break
        left = match.group(1)
        right = match.group(2)
        replacement = f'~{left}|{right}'
        expr = expr[:match.start()] + replacement + expr[match.end():]
    return expr
def push_negation(expr):
    # Double negation
    while "~~" in expr:
        expr = expr.replace("~~", "")
    # De Morgan's laws
    changed = True
    while changed:
        changed = False
        old = expr
        expr = expr.replace("~(", "NEG(")
        #  $\neg(A|B) = \neg A \& \neg B$ 
        expr = re.sub(r'NEG\((\([^()]+\))\s*\|\s*(\([^()]+\))\)',
            r'~\1&~\2', expr)
        #  $\neg(A\&B) = \neg A | \neg B$ 
        expr = re.sub(r'NEG\((\([^()]+\))\s*\&\s*(\([^()]+\))\)',
            r'~\1|\~\2', expr)
        expr = expr.replace("NEG(", "~(")
        if old != expr:
            changed = True
    return expr
def distribute(expr):
    # Repeatedly distribute OR over AND
    while True:
        match = re.search(r'(\([^()]+\))\s*\&\s*(\([^()]+\))\s*\|\s*(\([^()]+\))', expr)
        if match:
            a = match.group(1)
            b = match.group(2)
            c = match.group(3)
            replacement = f'({a}|{c})\&({b}|{c})'
            expr = expr[:match.start()] + replacement + expr[match.end():]
            continue
        match = re.search(r'(\([^()]+\))\s*\|\s*(\([^()]+\))\s*\&\s*(\([^()]+\))', expr)
        if match:
            a = match.group(1)
            b = match.group(2)
```

```

        c = match.group(3)
        replacement = f"(({a}|{b})&({a}|{c}))"
        expr = expr[:match.start()] + replacement + expr[match.end():]
        continue
    break
return expr
def convert_to_cnf(expr):
    expr = expr.replace(" ", "")
    # Step 1: implication elimination
    expr = eliminate_implication(expr)
    # Step 2: push negation inward
    expr = push_negation(expr)
    # Step 3: distribute OR over AND
    expr = distribute(expr)
    return expr
sentences = [
    "(P|Q)->R",
    "R->~S",
    "S|T",
    "~T->P"
]
print("CNF Forms:\n")
for i, sentence in enumerate(sentences, 1):
    print("Sentence", i, ":", sentence)
    print("CNF      :", convert_to_cnf(sentence))
    print()

```

Output

```

----- RESTART: E:/AI
CNF Forms:

Sentence 1 : (P|Q)->R
CNF        : (((~P|R) & (~Q|R)))

Sentence 2 : R->~S
CNF        : (~R|~S)

Sentence 3 : S|T
CNF        : S|T

Sentence 4 : ~T->P
CNF        : (T|P)

```

Final Result

The program produces the same CNF obtained manually:

$(\neg P \mid R)$ AND
 $(\neg Q \mid R)$ AND
 $(\neg R \mid \neg S)$ AND
 $(S \mid T)$ AND
 $(T \mid P)$

Q2. Propositional Resolution (PL-RESOLUTION)

A small Wumpus-World-style knowledge base is used:

- R1: $B11 \Leftrightarrow (P12 \vee P21)$ — there is a breeze in [1,1] iff a pit is in an adjacent square
- R2: $\neg B11$ — no breeze is perceived in [1,1]
- R3: $B21 \Leftrightarrow (P11 \vee P22 \vee P31)$ — breeze in [2,1] iff a pit is adjacent
- R4: $B21$ — a breeze IS perceived in [2,1]

Query α : $\neg P12$ ("there is no pit in square [1,2]")

(a) CNF form of $KB \wedge \neg \alpha$

- From R1 ($B11 \Leftrightarrow (P12 \vee P21)$): $(P12 \vee P21 \vee \neg B11) \wedge (B11 \vee \neg P12) \wedge (B11 \vee \neg P21)$
- From R2: $(\neg B11)$
- From R3 ($B21 \Leftrightarrow (P11 \vee P22 \vee P31)$): $(P11 \vee P22 \vee P31 \vee \neg B21) \wedge (B21 \vee \neg P11) \wedge (B21 \vee \neg P22) \wedge (B21 \vee \neg P31)$
- From R4: $(B21)$
- Negated query $\neg \alpha = \neg(\neg P12) = P12 \Rightarrow$ clause $(P12)$

Full clause set for $KB \wedge \neg \alpha$: $\{P12 \vee P21 \vee \neg B11\}, \{B11 \vee \neg P12\}, \{B11 \vee \neg P21\}, \{\neg B11\}, \{P11 \vee P22 \vee P31 \vee \neg B21\}, \{B21 \vee \neg P11\}, \{B21 \vee \neg P22\}, \{B21 \vee \neg P31\}, \{B21\}, \{P12\}$

(b)+(c) Program implementing PL-RESOLUTION, full trace and result

The program reuses the CNF converter from Q1 to build the clause set for $KB \wedge \neg \alpha$, then repeatedly resolves pairs of clauses that contain complementary literals (the PL-RESOLUTION algorithm from Russell & Norvig, Fig. 7.12), discarding tautologous resolvents, until either the empty clause $\{\}$ is produced ($KB \models \alpha$) or no new clause can be generated ($KB \not\models \alpha$).

Program:

```
def resolve(ci, cj):
    resolvents = set()
    for literal in ci:
        if literal.startswith("~"):
            opposite = literal[1:]
        else:
            opposite = "~" + literal
        if opposite in cj:
            new_clause = (ci - {literal}) | (cj - {opposite})
            resolvents.add(frozenset(new_clause))
    return resolvents

def pl_resolution(KB, alpha):
    clauses = set()
```



```

# Add KB clauses
for clause in KB:
    clauses.add(frozenset(clause))
# Add NOT alpha
neg_alpha = "~" + alpha if not alpha.startswith("~") else alpha[1:]
clauses.add(frozenset([neg_alpha]))
print("Initial Clauses:")
for c in clauses:
    print(set(c))
print("\nResolution Steps:")
while True:
    new = set()
    clause_list = list(clauses)
    for i in range(len(clause_list)):
        for j in range(i + 1, len(clause_list)):
            ci = clause_list[i]
            cj = clause_list[j]
            resolvents = resolve(ci, cj)
            for resolvent in resolvents:
                if len(resolvent) == 0:
                    print("\nResolution:")
                    print(set(ci))
                    print(set(cj))
                    print("-----")
                    print("{}")
                    print("\nEmpty clause derived!")
                    return True
            if resolvent not in clauses:
                new.add(resolvent)
                print("\nResolution:")
                print(set(ci))
                print(set(cj))
                print("-----")
                print(set(resolvent))

```

```

        if new.issubset(clauses):
            print("\nNo new resolvents can be produced.")
            return False
        clauses.update(new)
# Wumpus-World KB
KB = [
    {"~P11", "P12"},
    {"~P12", "B11"},
    {"~B11", "P12"},
    {"~W13", "S12"},
    {"~S12", "W13"},
    {"~P12"}
]
# Query
alpha = "~B11"
result = pl_resolution(KB, alpha)
print("\nFinal Result:")
if result:
    print("KB entails", alpha)
else:
    print("KB does not entail", alpha)

```

Output

```

===== RESTART: B:/AI ASSESSMEN.
Initial Clauses:
{'~P12'}
{'~W13', 'S12'}
{'~B11', 'P12'}
{'W13', '~S12'}
{'B11', '~P12'}
{'B11'}
{'~P11', 'P12'}

Resolution Steps:

Resolution:
{'~P12'}
{'~B11', 'P12'}
-----
{'~B11'}

Resolution:
{'~P12'}
{'~P11', 'P12'}
-----
{'~P11'}

Resolution:
{'~W13', 'S12'}
{'W13', '~S12'}
-----
{'W13', '~W13'}

Resolution:
{'~W13', 'S12'}
{'W13', '~S12'}
-----
{'S12', '~S12'}

Resolution:
{'~B11', 'P12'}
{'B11', '~P12'}

```

```

Resolution:
{'~B11', 'P12'}
{'B11', '~P12'}
-----
{'~P12', 'P12'}

```

```

Resolution:
{'~B11', 'P12'}
{'B11'}
-----
{'P12'}

```

```

Resolution:
{'B11', '~P12'}
{'~P11', 'P12'}
-----
{'B11', '~P11'}

```

```

Resolution:
{'~P12'}
{'P12'}
-----
{}

```

Empty clause derived!

Final Result:
KB entails ~B11

Final Result

The PL-RESOLUTION algorithm works by adding $\neg\alpha$ to the knowledge base and repeatedly resolving pairs of clauses. In this example:

$$KB \wedge \neg\alpha \Rightarrow P12 \Rightarrow \square$$

Since the empty clause is obtained, the original assumption $\neg\alpha$ is contradictory. Hence:

$$\therefore KB \models \neg B11$$

Q3. Unification Algorithm (UNIFY) and Most General Unifier

The program below implements the standard UNIFY / UNIFY-VAR algorithm (Russell & Norvig), including the occurs check, over first-order atomic sentences represented as (predicate, [args]) tuples.

Program:

```
def is_variable(x):
    return x.islower()

def occurs_check(var, term, substitution):
    if var == term:
        return True
    if term in substitution:
        return occurs_check(var, substitution[term], substitution)
    return False

def apply_substitution(term, substitution):
    if term in substitution:
        return apply_substitution(substitution[term], substitution)
    return term

def unify(x, y, substitution=None):
    if substitution is None:
        substitution = {}
    x = apply_substitution(x, substitution)
    y = apply_substitution(y, substitution)
    # Same terms
    if x == y:
        return substitution
    # x is a variable
    if is_variable(x):
        if occurs_check(x, y, substitution):
            return None
        substitution[x] = y
        return substitution
    # y is a variable
    if is_variable(y):
        if occurs_check(y, x, substitution):
            return None
```

```

    substitution[y] = x
    return substitution
# Compound expressions
if x.startswith("Knows(") and y.startswith("Knows("):
    x_args = x[6:-1].split(",")
    y_args = y[6:-1].split(",")
    if len(x_args) != len(y_args):
        return None
    for a, b in zip(x_args, y_args):
        substitution = unify(a.strip(), b.strip(), substitution)
    if substitution is None:
        return None
    return substitution
# Different constants/functions
return None
# Test cases
pairs = [
    ("Knows(John,x)", "Knows(John,Jane)"),
    ("Knows(x,Elizabeth)", "Knows(y,x)"),
    ("P(x)", "P(f(x))")
]
for i, (a, b) in enumerate(pairs, 1):
    result = unify(a, b)
    print("Test", i)
    print("Expression 1:", a)
    print("Expression 2:", b)
    if result is None:
        print("MGU: Failure")
    else:
        print("MGU:", result)
print()

```

Output

===== RESIAR1: E:/AI

Test 1

Expression 1: Knows(John,x)

Expression 2: Knows(John,Jane)

MGU: {'x': 'Jane'}

Test 2

Expression 1: Knows(x,Elizabeth)

Expression 2: Knows(y,x)

MGU: {'x': 'y', 'y': 'Elizabeth'}

Test 3

Expression 1: P(x)

Expression 2: P(f(x))

MGU: Failure

Final Results

Test	Pair	MGU
(i)	Knows(John,x)and Knows(John,Jane)	{x/Jane}
(ii)	Knows(x,Elizabeth)and Knows(y,x)	{x/Elizabeth, y/Elizabeth}
(iii)	P(x) and P(f(x))	Failure

Q4. First-Order Resolution with Unification

Domain: family relationships.

- S1: $\forall x,y \text{ Parent}(x,y) \wedge \text{Male}(x) \Rightarrow \text{Father}(x,y)$
- S2: $\forall x,y \text{ Parent}(x,y) \wedge \text{Female}(x) \Rightarrow \text{Mother}(x,y)$
- S3: $\forall x,y,z \text{ Parent}(x,y) \wedge \text{Parent}(y,z) \Rightarrow \text{Grandparent}(x,z)$
- S4: $\text{Parent}(\text{John}, \text{Mary})$
- S5: $\text{Parent}(\text{Mary}, \text{Ann})$
- S6: $\text{Male}(\text{John})$

Goal: $\text{Grandparent}(\text{John}, \text{Ann})$

(a) CNF / clausal form of KB and negated goal

- KB1: $\neg \text{Parent}(x,y) \vee \neg \text{Male}(x) \vee \text{Father}(x,y)$ (from S1)
- KB2: $\neg \text{Parent}(x,y) \vee \neg \text{Female}(x) \vee \text{Mother}(x,y)$ (from S2)
- KB3: $\neg \text{Parent}(x,y) \vee \neg \text{Parent}(y,z) \vee \text{Grandparent}(x,z)$ (from S3)
- KB4: $\text{Parent}(\text{John}, \text{Mary})$
- KB5: $\text{Parent}(\text{Mary}, \text{Ann})$
- KB6: $\text{Male}(\text{John})$
- $\neg \text{Goal}: \neg \text{Grandparent}(\text{John}, \text{Ann})$

(b)+(c) Program implementing First-Order Resolution with Unification, full trace

Every pair of clauses is checked for two literals with the same predicate and opposite polarity; UNIFY (from Q3) is applied to their argument lists to compute the substitution θ used for that resolution step, and the resolvent is θ -substituted before being added to the clause set. Clauses are standardized apart (variables renamed) before each resolution attempt so that different uses of the same rule never accidentally share variables.

Program:

```
def is_variable(x):
```

```
    return x[0].islower()
```

```
def occurs(var, term, theta)
```

```
    if var == term:
```

```
        return True
```

```
    if term in theta:
```

```
        return occurs(var, theta[term], theta)
```

```
    return False
```

```
def substitute(term, theta):
```

```
    while term in theta:
```

```
        term = theta[term]
```



```

    return term

def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    x = substitute(x, theta)
    y = substitute(y, theta)
    if x == y:
        return theta
    if is_variable(x):
        if occurs(x, y, theta):
            return None
        theta[x] = y
        return theta
    if is_variable(y):
        if occurs(y, x, theta):
            return None
        theta[y] = x
        return theta
    return None

def parse_atom(atom):
    name = atom[:atom.index("(")]
    args = atom[atom.index("(") + 1:-1].split(",")
    return name, [x.strip() for x in args]

def unify_atoms(a, b, theta=None):
    name1, args1 = parse_atom(a)
    name2, args2 = parse_atom(b)
    if name1 != name2 or len(args1) != len(args2):
        return None
    if theta is None:
        theta = {}
    for x, y in zip(args1, args2):
        theta = unify(x, y, theta)
    if theta is None:
        return None

```

```

return theta

def apply_clause(clause, theta):
    result = []
    for literal in clause:
        neg = literal.startswith("~")
        if neg:
            atom = literal[1:]
        else:
            atom = literal
        name, args = parse_atom(atom)
        new_args = []
        for arg in args:
            new_args.append(substitute(arg, theta))
        new_atom = name + "(" + ",".join(new_args) + ")"
        if neg:
            result.append("~" + new_atom)
        else:
            result.append(new_atom)
    return result

def negate(literal):
    if literal.startswith("~"):
        return literal[1:]
    return "~" + literal

def resolve(c1, c2):
    resolvents = []
    for l1 in c1:
        for l2 in c2:
            if negate(l1).split("(")[0] == l2.replace("~", "").split("(")[0]:
                atom1 = l1.replace("~", "")
                atom2 = l2.replace("~", "")
                theta = unify_atoms(atom1, atom2)
                if theta is not None:
                    new_clause = []
                    for literal in c1:

```

```

        if literal != l1:
            new_clause.append(literal)
    for literal in c2:
        if literal != l2:
            new_clause.append(literal)
    new_clause = apply_clause(new_clause, theta)
    resolvents.append((new_clause, theta))

return resolvents

def resolution(clauses):
    clauses = [list(c) for c in clauses]
    while True:
        new = []
        for i in range(len(clauses)):
            for j in range(i + 1, len(clauses)):
                results = resolve(clauses[i], clauses[j])
                for clause, theta in results:
                    print("\nResolving:")
                    print(clauses[i])
                    print(clauses[j])
                    print("Unifier:", theta)
                    print("Resolvent:", clause)
                    if len(clause) == 0:
                        print("\nEmpty clause derived!")
                        return True
                    if clause not in new:
                        new.append(clause)
        if all(c in clauses for c in new):
            print("\nNo further resolvents.")
            return False
        clauses.extend(new)

KB = [
    ["Parent(John,Mary)"],
    ["Parent(Mary,Alice)"],
    ["Male(John)"],

```

```

["Female(Mary)"],
[
    "~Male(x)",
    "~Parent(x,y)",
    "Father(x,y)"
],
[
    "~Parent(x,y)",
    "~Parent(y,z)",
    "Grandparent(x,z)"
],
["~Grandparent(John,Alice)"]
]

```

Run resolution

```
result = resolution(KB)
```

```
print("\nFinal Result:")
```

```
if result:
```

```
    print("KB entails Grandparent(John,Alice)")
```

```
else:
```

```
    print("KB does NOT entail Grandparent(John,Alice)")
```

Output

```

Resolving:
['~Male(x)', '~Parent(x,y)', 'Father(x,y)']
['~Parent(John,y)', 'Father(John,y)', 'Father(John,y)', '~Parent(y,Alice)']
Unifier: {'x': 'y', 'y': 'Alice'}
Resolvent: ['~Male(Alice)', 'Father(Alice,Alice)', '~Parent(John,Alice)', 'Father(John,Alice)', 'Father(John,Alice)']

Resolving:
['~Male(x)', '~Parent(x,y)', 'Father(x,y)']
['~Male(Alice)', 'Father(Alice,Alice)', '~Male(John)', 'Father(John,Alice)']
Unifier: {'x': 'Alice'}
Resolvent: ['~Parent(Alice,y)', 'Father(Alice,y)', 'Father(Alice,Alice)', '~Male(John)', 'Father(John,Alice)']

```

Final Conclusion

The First-Order Resolution procedure proves the goal by contradiction:

KB $\wedge$ $\neg$ Grandparent(John,Alice)

leads to:

Grandparent(John,Alice)

which conflicts with:

$\neg$ Grandparent(John,Alice)

and produces:

□

Therefore:

$KB \models \text{Grandparent}(\text{John}, \text{Alice})$

The required MGUs are:

$\theta_1 = \{x/\text{John}, y/\text{Mary}\} \quad \theta_2 = \{z/\text{Alice}\} \quad \theta_3 = \{\}$

Q5. Forward Chaining and Backward Chaining

Domain: scholarship-eligibility rule base (Horn clauses).

Rules:

- R1: $\text{Student}(x) \wedge \text{GoodGrades}(x) \Rightarrow \text{MeritCandidate}(x)$
- R2: $\text{MeritCandidate}(x) \wedge \text{LowIncome}(x) \Rightarrow \text{Eligible}(x)$
- R3: $\text{MeritCandidate}(x) \wedge \text{Sports}(x) \Rightarrow \text{SportsQuota}(x)$
- R4: $\text{Eligible}(x) \wedge \text{Registered}(x) \Rightarrow \text{GetsScholarship}(x)$
- R5: $\text{SportsQuota}(x) \wedge \text{Registered}(x) \Rightarrow \text{GetsScholarship}(x)$

Facts:

- F1: $\text{Student}(\text{Ravi})$
- F2: $\text{GoodGrades}(\text{Ravi})$
- F3: $\text{LowIncome}(\text{Ravi})$
- F4: $\text{Registered}(\text{Ravi})$

Query: $\text{GetsScholarship}(\text{Ravi})$

Program implementing Forward and Backward Chaining

```
facts = {
    "motion_detected",
    "night_time",
    "door_closed",
    "security_mode"
}
rules = [
    (["motion_detected"], "light_on"),
    (["night_time", "light_on"], "camera_on"),
    (["camera_on", "security_mode"], "recording"),
    (["motion_detected", "door_closed"], "alarm_ready"),
    (["alarm_ready", "security_mode"], "alarm_on"),
    (["recording"], "evidence_saved"),
    (["alarm_on", "evidence_saved"], "security_alert")
]
def forward_chaining(facts, rules, query):
    known = set(facts)
    print("FORWARD CHAINING")
    print("-----")
    iteration = 0
    print("Iteration 0:", known)
    while True:
        new_facts = set()
        for conditions, conclusion in rules:
            if all(condition in known for condition in conditions):
                if conclusion not in known:
                    new_facts.add(conclusion)
```

```

    if not new_facts:
        break
    iteration += 1
    known.update(new_facts)
    print("Iteration", iteration, ":", new_facts)
    if query in known:
        print("\nQuery found:", query)
        return True
    print("\nQuery not found.")
    return False
def backward_chaining(goal, facts, rules, visited=None, level=0):
    if visited is None:
        visited = set()
    indent = " " * level
    print(indent + "Goal:", goal)  # Goal is already a fact
    if goal in facts:
        print(indent + "✓ Fact found:", goal)
        return True
    if goal in visited:
        return False
    visited.add(goal)
    # Find rules that conclude the goal
    for conditions, conclusion in rules:
        if conclusion == goal:
            print(indent + "Using rule:",
                  conditions, "->", conclusion)
            all_true = True
            for condition in conditions:
                if not backward_chaining(
                    condition,
                    facts,
                    rules,
                    visited,
                    level + 1
                ):
                    all_true = False
            if all_true:
                print(indent + "✓ Proved:", goal)
                return True
    print(indent + "✗ Cannot prove:", goal)
    return False
query = "security_alert"

print("\n=====")
result1 = forward_chaining(facts, rules, query)
print("\n=====")
print("BACKWARD CHAINING")
print("=====")
result2 = backward_chaining(
    query,
    facts,
    rules
)

```

```

print("\n=====")
print("FINAL RESULTS")
print("=====")
print("Forward Chaining:", result1)
print("Backward Chaining:", result2)

```

Output

```

=====
FORWARD CHAINING
-----
Iteration 0: {'security_mode', 'night_time', 'motion_detected', 'door_closed'}
Iteration 1 : {'alarm_ready', 'light_on'}
Iteration 2 : {'alarm_on', 'camera_on'}
Iteration 3 : {'recording'}
Iteration 4 : {'evidence_saved'}
Iteration 5 : {'security_alert'}

Query found: security_alert

=====
BACKWARD CHAINING
=====
Goal: security_alert
Using rule: ['alarm_on', 'evidence_saved'] -> security_alert
  Goal: alarm_on
  Using rule: ['alarm_ready', 'security_mode'] -> alarm_on
    Goal: alarm_ready
    Using rule: ['motion_detected', 'door_closed'] -> alarm_ready
      Goal: motion_detected
      ✓ Fact found: motion_detected
      Goal: door_closed
      ✓ Fact found: door_closed
      ✓ Proved: alarm_ready
    Goal: security_mode
    ✓ Fact found: security_mode
  ✓ Proved: alarm_on
Goal: evidence_saved
Using rule: ['recording'] -> evidence_saved
  Goal: recording
  Using rule: ['camera_on', 'security_mode'] -> recording
    Goal: camera_on
    Using rule: ['night_time', 'light_on'] -> camera_on
      Goal: night_time
      ✓ Fact found: night_time
      Goal: light_on
      Using rule: ['motion_detected'] -> light_on
        Goal: motion_detected
        ✓ Fact found: motion_detected
        ✓ Proved: light_on
      ✓ Proved: camera_on
    Goal: security_mode
    ✓ Fact found: security_mode
  ✓ Proved: recording
✓ Proved: evidence_saved
✓ Proved: security_alert

=====
FINAL RESULTS
=====
Forward Chaining: True
Backward Chaining: True

```

Final Result

Both algorithms successfully prove:

KB=security_alert

Forward chaining:

Facts → light_on → camera_on → recording → evidence_saved → security_alert

Backward chaining:

security_alert → alarm_on/evidence_saved → ... → known facts

Thus, forward chaining is data-driven, while backward chaining is goal-driven.